ARCHITECTURE & DESIGN

HORIZON GRID

Technical Architecture

Version: 0.2.5

Documentation prepared: 2026-08-23

PREPARED FOR EVALUATION

Table of Contents

Technical Architecture Overview

HORIZON GRID is a self-hosted tool that looks up an **IOC** (Indicator of Compromise — e.g. an IP address, domain, URL, or file hash) against many third-party threat-intelligence sources at once, then uses an AI model to summarize and correlate the results. The backend's own self-description (`backend/app/main.py`) puts it this way:

> *"single-search IOC lookup across dozens of providers, correlated and summarized by a local Ollama model (or AWS Bedrock/Gemini/Anthropic)."*

This section describes what the platform is built from at the container/process level, not the request-level data flow (covered elsewhere in this document) or the AI/provider internals (also covered elsewhere).

Container Inventory

`docker-compose.yml` defines seven backing services — `postgres` , `redis` , `neo4j` , `opensearch` , `backend` , `celery_worker` , and `celery_beat` — plus the `frontend` container, for eight containers running the full stack.

Container	Image / stack	Role	Implementation status
`postgres`	`postgres:16-alpine`	System of record: lookups, provider results, AI summaries, correlation edges, evidence, cases, users.	Actively used — the only datastore actually written to during a lookup.
`redis`	`redis:7-alpine`	Provider-result cache (logical DB `/0`), per-user rate limiter, Celery message broker (`/1`) and Celery result backend (`/2`).	Actively used.
`neo4j`	`neo4j:5-community` + APOC plugin	Config field and container exist to "mirror" correlation edges into a graph store, per two module docstrings.	Provisioned only — no driver instantiation, Cypher query, or read/write call exists anywhere in the backend code. Not used for anything today.
`opensearch`	`opensearchproject/opensearch:2.17.0` (`DISABLE_SECURITY_PLUGIN: "true"`)	Config field and container exist for an intended full-text/search index.	Provisioned only — no indexing or search client code exists in the backend. No search functionality is implemented against it.
`backend`	FastAPI, Python 3.12	Core API server: IOC-type detection, provider orchestration, AI summarization/correlation, evidence building, auth, case management.	Actively used — the primary application process.
`celery_worker`	Celery 5.4 (same backend codebase)	Executes background jobs.	Actively used, but for exactly one job (see below).
`celery_beat`	Celery 5.4 beat scheduler	Triggers scheduled jobs on a timer.	Actively used, one scheduled job only.
`frontend`	Next.js 14.2.15	Web UI.	Actively used.

A second compose file, `docker-compose.prod.yml`, is layered on top of `docker-compose.yml` for production use (this is what the Windows installer runs): it strips the development bind-mounts and switches the `backend` / `frontend` containers to their production start commands rather than dev/hot-reload ones.

Neo4j and OpenSearch, specifically: both are real, running containers in every deployment of this stack, and both have a corresponding settings field in the backend config (`neo4j_uri` / `neo4j_user` / `neo4j_password`, `opensearch_url`). Neither is wired into any actual code path. The correlation graph that the product presents to users lives entirely in Postgres's `correlation_edges` table today. Any description of a "Neo4j-backed graph" or "OpenSearch-backed search" elsewhere should be read as an architecturally-provisioned but not-yet-integrated capability, not a working feature.

Backend

The backend is a **FastAPI** application (`backend/app/main.py`) running on **Python 3.12**. Its notable stack choices:

- **SQLAlchemy 2.0**, async, with `asyncpg` as the Postgres driver.
- **Alembic** for schema migrations (`backend/alembic/`).
- **Celery 5.4** for the one background job described below.
- **structlog** and **prometheus-fastapi-instrumentator** for logging and metrics.

The backend is also where the 16 third-party data-source connectors ("providers"), the AI summarization/correlation logic, and the authentication/RBAC (role-based access control) system live — each of those is detailed in its own section of this document.

Frontend

The frontend is a **Next.js 14.2.15** application using **React 18.3.1** and **TypeScript**, styled with **Tailwind**, with **Zustand** for client-side state, **Recharts** for charts, **react-force-graph-2d** for the relationship/correlation graph visualization, and **Radix UI** for primitive UI components (all per `frontend/package.json`).

`package.json` declares `vitest` as the test runner (`"test": "vitest run"`), but a repository-wide search found zero `*.test.*` / `*.spec.*` files under `frontend/` — the tooling is configured but no frontend automated tests currently exist. (See the Testing and Quality Assurance appendix for the full picture, including the backend's test suites.)

Executive Dashboard and Deterministic Scoring Engine

A new **Executive Dashboard** (`GET /api/v1/dashboard/kpis`, backed by `app/core/dashboard.py::get_kpis()`) gives an at-a-glance operational view of the whole platform: real-time KPI tiles for active investigations, the count of critical/high-risk IOCs, open case count and open-critical case count (reported as two separate numbers, not collapsed into one), average threat score, provider health percentage, and AI analysis success rate. Every one of these is a live SQL aggregate computed against Postgres at request time — none is hardcoded, and none is cached client-side across page loads. A companion endpoint, `GET /api/v1/dashboard/executive-summary` (`app/ai/dashboard_summary.py`), layers an AI-generated 2–4 sentence narrative on top of the exact same KPI values, grounded exclusively in those numbers as given facts. If the AI backend is unreachable, or its structured response fails validation twice in a row, the endpoint falls back to a deterministic, template-generated sentence built directly from the same real KPI numbers — never an error message, and never fabricated data. This is disclosed to the user directly on the Dashboard itself: the Executive Summary card carries an "AI-generated" badge when the narrative came from the model, or a "Template fallback" badge when it didn't, so the source of the text is never ambiguous.

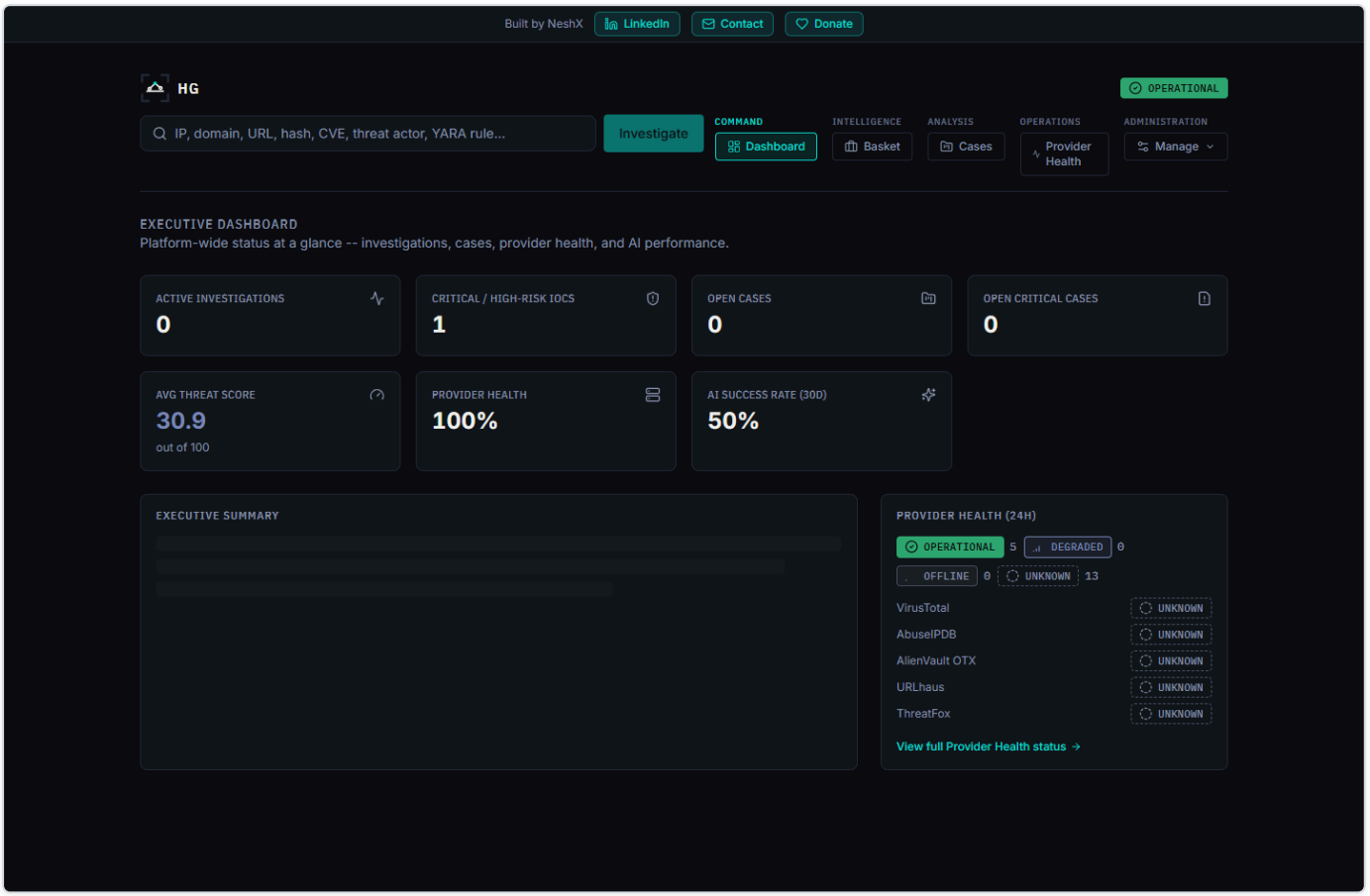

Figure 1 — The Executive Dashboard shows real KPI tiles, an AI-generated (or template-fallback) executive summary, and a provider-health summary widget -- every number sourced live, never hardcoded.

Underlying both the Dashboard's "average threat score" tile and every individual investigation's risk score is a new **deterministic (non-AI) scoring engine** (`app/scoring/engine.py`). Historically, `overall_risk_score` , `confidence_score` , `malicious_probability` , and the final verdict were 100% AI-generated in one call — prompt wording alone cannot reliably stop a model from overriding a number it was merely asked not to touch. The scoring engine instead computes all four values *before* any AI call runs, purely from already-fetched evidence: a cross-provider verdict-consensus component (weighted 65 of 100 points, with a corroboration multiplier so a single unconfirmed provider flag scores meaningfully lower than several providers agreeing) and a correlation-graph-evidence component (35 of 100 points, drawn from `app/correlation/engine.py` 's graph and given the same corroboration treatment to resist a single source flooding it with distinct, uncorroborated edges). The AI is then handed this already-decided score as a given fact and asked only to narrate a verdict/rationale consistent with it — it can never invent or override the number, and the platform's own final-assessment validation re-checks the AI's output against the deterministic result before persisting it. Every persisted assessment records `SCORING_ENGINE_VERSION` , the exact version of this weighting scheme that produced it, so any historical score can always be traced back to the logic that generated it.

Celery: Scope and a Key Boundary

`celery_worker` and `celery_beat` both run the same task application, `app.workers.celery_app` . Exactly **one** scheduled task exists in the codebase: `run_osint_crawl` , triggered hourly by `celery_beat` (`backend/app/workers/celery_app.py`), which re-crawls OSINT (open-source intelligence) sources for IOCs that were looked up in the last 24 hours (`backend/app/workers/tasks.py`).

That is the entirety of what Celery does in this platform. **The main, user-facing IOC lookup pipeline — the synchronous, streamed lookup reachable at `POST /api/v1/lookup/stream` — does not go through Celery at all.** This is stated directly in a header

comment in `celery_app.py`. A lookup request (provider fan-out, per-provider AI summarization, correlation, final AI assessment, and persistence) is handled entirely within the FastAPI backend process for the duration of that one HTTP connection, with results streamed to the browser as Server-Sent Events (SSE) as each step completes. Celery is reached only by the separate, unrelated hourly re-crawl job — never as part of answering a live lookup request.

Deployment Paths

Two deployment paths are implemented and verified in the repository:

1. **Docker Compose** — `docker-compose.yml` (development) and `docker-compose.yml` + `docker-compose.prod.yml` (production). This is the path the Windows installer automates end-to-end (covered in the Windows Deployment / Installer section of this document).
2. **Kubernetes** — the `k8s/` directory contains a parallel, **Kustomize**-based Kubernetes deployment: StatefulSets for `postgres`, `neo4j`, and `opensearch`; Deployments for `backend`, `frontend`, and `celery`; and an Ingress resource. This is an alternative to the Docker Compose / Windows-installer path. Beyond its existence and the resource types listed above, further detail on this path (replica counts, resource limits, ingress rules, or how it is intended to be operated) is **Not confirmed** — it was not otherwise inspected as part of this documentation effort.

Component Diagram

The diagram below reflects verified relationships only. Solid arrows are real, active data paths confirmed in code; dashed arrows into `neo4j` and `opensearch` represent the config fields and running containers that exist but have no consuming application code.

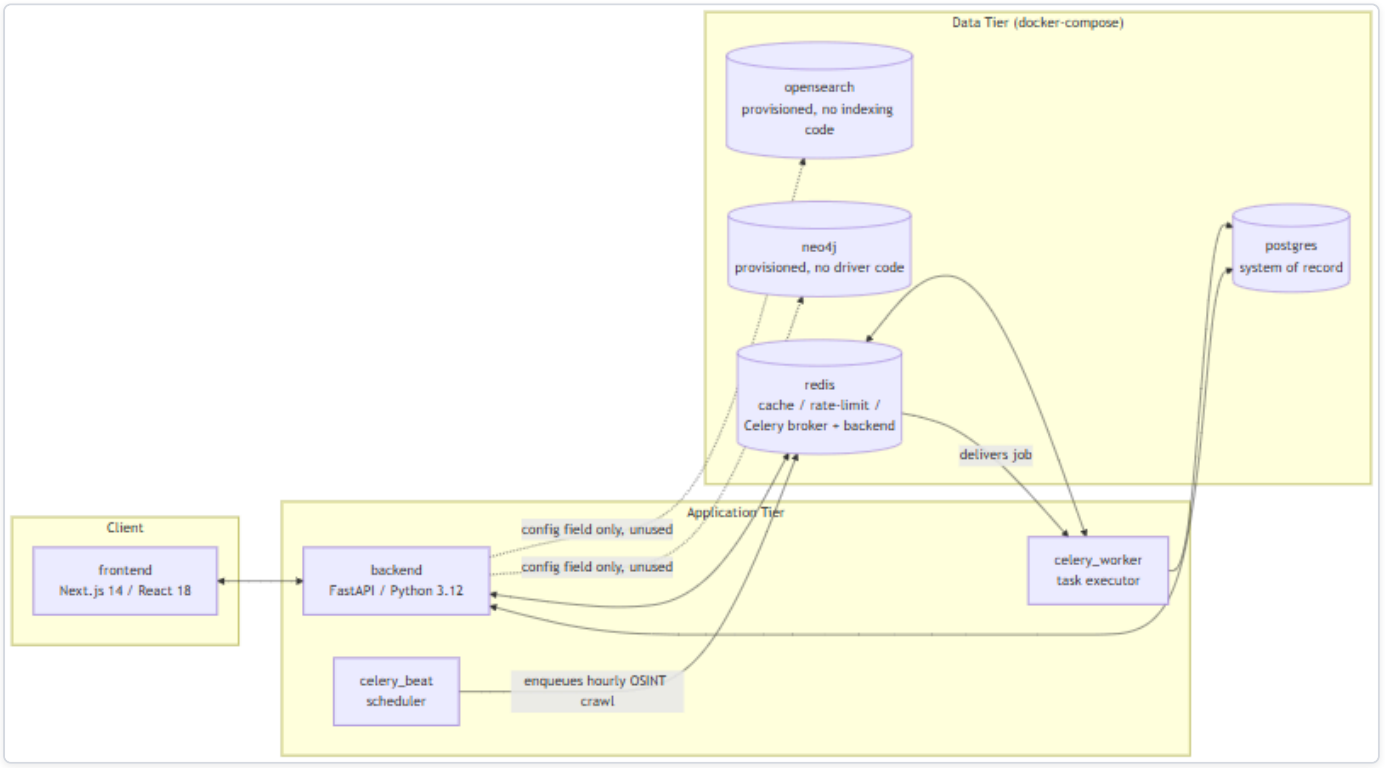

Figure 2 — Diagram: Component Diagram

Note what is *not* in this diagram: there is no edge from `backend` to `celery_worker` / `celery_beat` for the lookup pipeline, because none exists. The live IOC-lookup path is entirely `frontend <-> backend <-> postgres/redis`; the Celery containers are reached only

by the independent, hourly-scheduled OSINT re-crawl job described above.

Data Flow

This section traces exactly what happens, in code, when a user submits one IOC (Indicator of Compromise — a piece of evidence such as an IP address, domain, URL, file hash, CVE ID, or MITRE ATT&CK technique ID that a security analyst wants to investigate) to the platform, from the initial HTTP request through to the final "done" event. It then states plainly which of the platform's four datastores actually participate in that flow today, and which do not.

The entry point for a lookup is a single endpoint: `POST /api/v1/lookup/stream` (`backend/app/api/routes/lookup.py`). It is a Server-Sent Events (SSE) endpoint — the client opens one HTTP connection and receives a sequence of typed events as the lookup progresses, rather than blocking for a single response. The sequence below is verified directly against that endpoint's implementation.

The 9-Step Lookup Pipeline

1. **Rate limit check.** A Redis-backed fixed-window rate limiter (`RateLimiter` , `app/core/cache.py`) rejects the request if the calling user has exceeded the configured quota — 10 calls per 60 seconds per user by default (`app/core/config.py`), both values configurable.
2. **IOC type detection.** `detect_ioc_type()` (`app/ioc/detector.py`) classifies the raw input string using ordered regex and heuristic checks into one of 32 `IOCType` enum values (`app/ioc/types.py`) — for example, distinguishing an IPv4 address from a domain from a SHA256 hash.
3. **Lookup record created.** An `IOCLookup` row is created in Postgres immediately, with `status = RUNNING` . This is the row that will later be updated with the final verdict and risk scores.
4. **Provider fan-out.** `run_all_providers()` (`app/providers/orchestrator.py`) concurrently invokes every registered provider (of 16 total) whose `supports(ioc_type)` returns true for this IOC's type, using `asyncio.create_task` and `asyncio.as_completed` — providers run in parallel, not sequentially. A "provider" here is an external threat-intelligence, vulnerability, or OSINT source the platform queries (for example VirusTotal, AbuseIPDB, NVD). Each provider call passes through a Redis cache check first, then `BaseProvider.run()` (timeout and retry handled by `tenacity` , with configurable `provider_timeout_seconds` and `provider_max_retries`), producing a normalized `ProviderResult` .
5. **Per-provider persistence and summary.** As each provider result arrives (not waiting for all of them), it is persisted to Postgres as a `ProviderResultRecord` . If that provider's status is `ok` , `summarize_provider()` (`app/ai/service.py`) makes one AI call for that provider alone, grounded only in that provider's own JSON response, and the result is persisted as an `AISummaryRecord` tagged with that `provider_id` . Both the raw result and the summary are streamed to the client as SSE events (`provider_result` , `provider_summary`) and committed to Postgres after every single provider completes — a deliberate design so that a client disconnecting mid-lookup does not lose already-completed provider data (`lookup.py` , lines 122-136).
6. **Correlation.** Once every provider has finished, `correlate()` (`app/correlation/engine.py`) runs. This is a pure, I/O-free function: it extracts typed relationship edges from normalized provider fields (`resolved_ips` , `related_hashes` , `malware_families` , `mitre_techniques` , `cves` , and others), deduplicates and corroborates them — each additional provider that independently reports the same fact boosts its confidence score by a fixed increment (`_CORROBORATION_BONUS_PER_PROVIDER = 0.15`) — and returns a `CorrelationResult` (nodes, edges, deduplicated facts, provider agreement). The resulting edges are persisted as `CorrelationEdgeRecord` rows in Postgres and streamed as a `correlation` SSE event.
7. **Final AI assessment.** `generate_final_assessment()` (`app/ai/service.py`) makes exactly one consolidated AI call, grounded in all of the per-provider summaries plus the correlation output — never in the raw provider JSON directly. The result is validated against the `FinalAssessment` schema, cross-checked against the deterministic correlation data by `_ground_final_assessment()` (which silently strips any agreeing/disagreeing provider citation that isn't actually backed by real correlation data, and separately

flags — rather than removes — any cited MITRE technique that isn't backed by a real correlation edge, by setting that technique mapping's `grounded` field to `False`), and written onto the `IOCLookup` row as `final_verdict`, `risk_score`, `confidence_score`, and the full `final_assessment` JSON.

8. **Evidence ledger build.** `build_evidence()` (`app/evidence/builder.py`) deterministically converts the provider results, AI summaries, and correlation edges into `EvidenceItem` rows. This step is explicitly **not AI-generated** (per the module's own docstring) — it exists so that later AI-driven explanations elsewhere in the product can cite real, checkable evidence records instead of re-summarizing from memory.
9. **Final SSE events.** The stream emits a `final_assessment` event, then a `done` event, closing the connection.

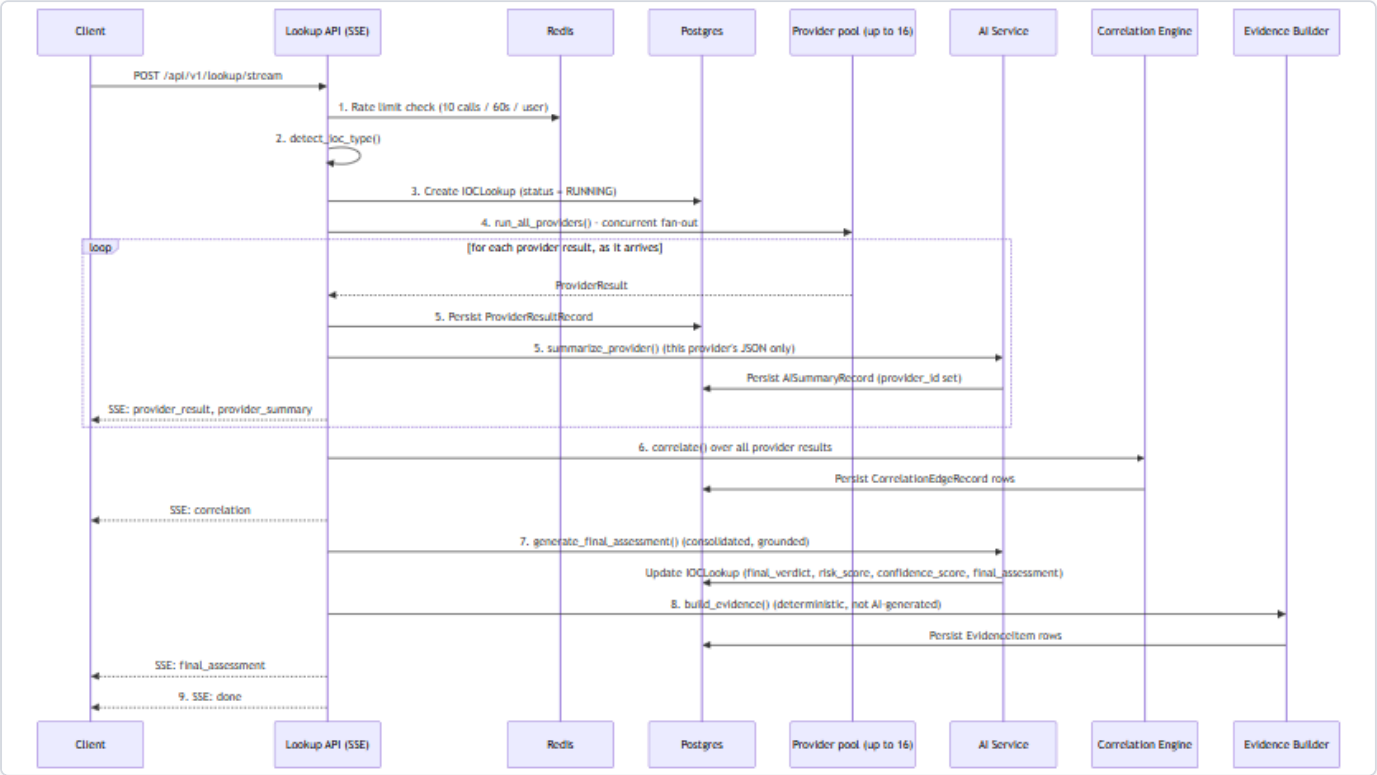

Figure 3 — Diagram: The 9-Step Lookup Pipeline

Where Data Actually Lives

The Docker Compose stack provisions four datastores — Postgres, Redis, Neo4j, and OpenSearch. Reading the pipeline above, it would be easy to assume all four are in play. They are not. This distinction matters enough that it is called out separately: **only Postgres and Redis have any application code that actually reads from or writes to them.** Neo4j and OpenSearch are running containers with nothing in the backend that talks to them.

Datastore	Provisioned in `docker-compose.yml`?	Application code that uses it?	What it actually holds today
Postgres	Yes (`postgres:16-alpine`)	Yes	Everything: `IOCLookup` rows, `ProviderResultRecord`s, `AISummaryRecord`s (per-provider and final), `CorrelationEdgeRecord`s, `EvidenceItem`s, plus users, cases, and analyst baskets
Redis	Yes (`redis:7-alpine`)	Yes	Provider-result cache (keyed `provider_cache:{provider_id}:{ioc_type}:{sha256(value)}`, default TTL 3600s), the lookup-creation rate limiter, and the Celery broker/result-backend queues
Neo4j	Yes (`neo4j:5-community` + APOC plugin)	**No**	Nothing. Config fields only (`neo4j_uri`, `neo4j_user`, `neo4j_password`)
OpenSearch	Yes (`opensearchproject/opensearch:2.17.0`)	**No**	Nothing. A config field only (`opensearch_url`)

Postgres is the correlation graph today

Step 6 of the pipeline above talks about a "correlation graph" of nodes and edges between IOCs. It is tempting to assume that graph lives in Neo4j, since Neo4j is a graph database and is sitting right there in the stack. It does not. **The correlation graph is a set of rows in Postgres's `correlation_edges` table, queried relationally — full stop.** `CorrelationEdgeRecord`'s own docstring (`models/lookup.py`) and the correlation engine's module docstring (`correlation/engine.py`) both describe edges as being "mirrored into Neo4j," but that describes an intended future design, not current behavior. A full-repo search of `backend/app` for `neo4j`, `Neo4j`, or `GraphDatabase` (outside the Python virtual environment) turns up zero driver instantiations, zero Cypher queries, and zero read or write calls anywhere in the application. The only things Neo4j-related in the codebase are the three unused config fields and those two docstrings.

There is no search functionality

The same is true of OpenSearch, with even less ambiguity: there are no docstrings suggesting an intended integration, no indexing pipeline, and a full-repo search finds zero OpenSearch client, index, or search code anywhere in `backend/app`. The container runs; nothing in the product talks to it. Any full-text search a user might expect (e.g., searching historical lookups by free text) is not implemented against OpenSearch or anywhere else.

Practical takeaway

For a judge or engineer evaluating this build: treat Neo4j and OpenSearch as **provisioned infrastructure for a future phase**, not as working features. If asked "does this platform have a graph database backing its correlation view?" or "does it support full-text search?", the accurate answer is no — both containers exist and are healthy, but the entire correlation and evidence pipeline described in the nine steps above runs, end to end, on Postgres and Redis alone.

Database Architecture

HORIZON GRID's `docker-compose.yml` provisions four datastore containers: `postgres` (postgres:16-alpine), `redis` (redis:7-alpine), `neo4j` (neo4j:5-community with the APOC plugin), and `opensearch` (opensearchproject/opensearch:2.17.0). This section describes what each one actually does in the running system, based on direct inspection of the backend code rather than the docker-compose comments or model docstrings that describe the *intended* design. Two of the four -- Neo4j and OpenSearch -- are running containers with no backend code that reads from or writes to them; that gap is documented plainly below rather than glossed over.

An **IOC** (Indicator of Compromise) is the value a user submits for lookup -- an IP address, domain, file hash, URL, CVE ID, etc. A **provider** is a connector to an external threat-intelligence source (VirusTotal, AbuseIPDB, and so on) that the platform queries for a given IOC.

PostgreSQL: the system of record

PostgreSQL is the only datastore the application actually writes to during a lookup. Every table below is defined under `backend/app/models/`:

Table	Holds
`users`	Account records: email, hashed password, full name, role (`admin` / `analyst` / `viewer`), active flag.
`ioc_lookups`	One row per lookup: the submitted IOC value and type, a status enum (`pending` / `running` / `completed` / `failed`), and the final AI output -- `final_verdict` , `risk_score` , `confidence_score` , and the full `final_assessment` as JSONB.
`provider_results`	One row per provider call per lookup: that provider's raw response as JSONB, plus its status and latency.
`ai_summaries`	AI-generated summaries as JSONB. A nullable `provider_id` distinguishes a per-provider summary from the single final consolidated assessment (null `provider_id` marks the latter).
`correlation_edges`	The relationship graph produced by the correlation engine: source/target type and value, relationship type, a confidence score, and provenance (which provider(s) contributed the edge).
`evidence_items`	The deterministic, non-AI-generated evidence ledger: an evidence-type enum (9 values), a source label, a claim, an interpretation, a confidence score (0-100), and the underlying raw data as JSONB.
`basket_items`	Per-analyst scratch lists of IOCs, unique per owner and IOC value.
`cases` , `case_iocs` , `case_notes` , `case_reports`	A lightweight case-management workflow: cases with status/severity enums, the IOCs attached to a case, and analyst notes. `case_reports` exists as a table, but report generation itself is not implemented -- the field is present and permanently empty rather than populated by any code path.

Two things worth calling out because they matter for how the rest of this appendix should be read:

- **The correlation graph lives in Postgres.** `correlation_edges` is a plain relational table, populated by the correlation engine after every provider has returned, and streamed to the UI as part of the lookup. There is no separate graph database behind it today -- see the Neo4j section below.
- **The evidence ledger is deliberately not AI-generated.** `evidence_items` rows are built by a deterministic conversion step so that later AI-written explanations have real, checkable records to cite against, rather than citing their own prior output.

Schema changes are managed with **Alembic** (`backend/alembic/`), SQLAlchemy's migration tool. The application itself uses SQLAlchemy 2.0 in async mode over `asyncpg` .

Neo4j: provisioned, not integrated

Neo4j (`neo4j:5-community` , with the APOC plugin) runs as a container in `docker-compose.yml` , and the backend config (`app/core/config.py`) has `neo4j_uri` , `neo4j_user` , and `neo4j_password` settings for it. Some code comments describe an intended design where correlation edges get "mirrored into Neo4j" -- this appears in the `CorrelationEdgeRecord` model docstring and in the correlation engine's module docstring.

That design was never built. A full search of the backend application code for `neo4j` , `Neo4j` , or `GraphDatabase` (excluding the Python virtual environment) turns up zero driver instantiation, zero Cypher queries, and zero read or write calls anywhere in the application. Neo4j is not used for anything today. **The correlation graph described above is, in its entirety, the `correlation_edges` table in PostgreSQL.** Anyone evaluating this platform should treat Neo4j as reserved-but-idle infrastructure, not as an active graph store.

OpenSearch: provisioned, not integrated

The same pattern holds for OpenSearch (`opensearchproject/opensearch:2.17.0` , running with `DISABLE_SECURITY_PLUGIN: "true"`). The backend config has an `opensearch_url` setting, and the container runs as part of the stack, but a full search of the backend application code found zero OpenSearch client code, zero index definitions, and zero search calls. No indexing pipeline exists, and no user-facing search feature is backed by it. OpenSearch should be described as provisioned infrastructure only -- not as an active search index.

Neo4j and OpenSearch represent the most significant gap between the design implied by the docker-compose file and docstrings, and what the running system actually does. Both are real containers consuming real resources; neither currently does any work for the platform.

Redis: three real uses

Unlike Neo4j and OpenSearch, Redis (`redis:7-alpine`) is genuinely load-bearing, in exactly three ways -- all confirmed against `app/core/cache.py` , `docker-compose.yml` , and `app/core/config.py` :

1. **Provider-result cache.** Each provider call is cached under a key of the form `provider_cache:{provider_id}:{ioc_type}:{sha256(value)}` , with a TTL controlled by `provider_cache_ttl_seconds` (default 3600 seconds / 1 hour). A repeat lookup for the same IOC within the TTL window skips the live provider call.
2. **Rate limiter.** A fixed-window rate limiter, implemented in the same `app/core/cache.py` module as the cache, guards the lookup-creation endpoint: by default, 10 calls per 60 seconds per user (both values configurable).
3. **Celery broker and result backend.** The `celery_worker` / `celery_beat` services use Redis as both the message broker and the result store for background jobs.

Redis is used across separate **logical database indexes** on the same Redis instance, rather than one shared keyspace:

Redis DB index	Purpose
`/0`	Cache (provider-result cache and rate-limiter counters, both via `app/core/cache.py`)
`/1`	Celery broker (`redis://redis:6379/1`)
`/2`	Celery result backend (`redis://redis:6379/2`)

This separation means a `FLUSHDB` or eviction event against the cache database, for example, cannot collide with in-flight Celery task state, and vice versa -- they are isolated by index even though they share one Redis process. No other use of Redis was found in the

codebase.

It's worth noting that the synchronous SSE lookup pipeline (`POST /api/v1/lookup/stream`) does **not** go through Celery at all -- it runs providers directly and streams results back over the same request. The only scheduled Celery job in the system is an hourly OSINT re-crawl (`run_osint_crawl`) for IOCs looked up in the last 24 hours; Celery's broker/result-backend role is scoped to that background job, not to the interactive lookup path.

Component overview

Figure 4 — Diagram: Component overview

Summary

Postgres is the single source of truth for every entity in the platform, including the correlation graph. Redis does three concrete jobs -- provider-result caching, rate limiting, and Celery transport -- cleanly separated by logical database index on one instance. Neo4j and OpenSearch run as containers and appear in configuration, but as of this writing carry no application code that reads or writes to them; they should be read as forward-provisioned infrastructure, not as active components of the current architecture.

Linux Packaging Architecture

Application layer: unchanged

HORIZON GRID's application layer is identical on Linux and Windows. Both platforms run the same Docker Compose stack -- Postgres, Redis, Neo4j, OpenSearch, the FastAPI backend, the Next.js frontend, and two Celery workers -- built from the same `backend/`, `frontend/`, `docker-compose.yml`, and `docker-compose.prod.yml`. Neither platform bundles Node/pip dependencies or pre-built images in its installer artifact; both install them the same way, inside the containers, on the first `docker compose up --build`. Both platforms' Compose project directory shares the basename `app`, so both produce the identical Docker Compose project label (`com.docker.compose.project=app`). There is no architectural divergence at this layer -- it is the same code, the same images, the same runtime behavior.

Installer/lifecycle layer: the only difference

All Linux-specific work is confined to the outer packaging and lifecycle-management layer, which replaces the Windows installer stack component-for-component:

Windows	Linux
Inno Setup installer	`.deb` package (`horizon-grid_0.2.0_amd64.deb`)
WinForms setup wizard	Python CLI setup wizard (terminal-based, `horizon-grid configure`)
Windows service registration	systemd unit (`horizon-grid.service`)
Start Menu shortcut group	`horizon-grid` CLI entrypoint + desktop menu launcher
ProgramData config (ACL-locked)	`/etc/horizon-grid/.env` (root:root, chmod 600)

The CLI wizard (`linux/wizard/setup_wizard.py`) follows the same flow and issues the same HTTP calls as its Windows counterpart (`windows/wizard/Setup-Wizard.ps1`): Welcome -> Administrator Account -> AI Configuration -> Threat Intelligence Providers -> Network Ports -> Summary -> write `.env` -> `docker compose up -d --build` -> wait for `/health` -> register the admin account -> confirm login. The systemd unit is registered at install time but never auto-started -- nothing runs until the admin completes the wizard, matching the Windows installer's own "nothing auto-starts before configuration" behavior.

Installed file layout (FHS)

Path	Purpose	Windows analog
`/opt/horizon-grid/app/`	Application code (backend/, frontend/, compose files, docs)	Program Files
`/etc/horizon-grid/.env`	Config/secrets, root:root, chmod 600	ProgramData config (ACL-locked)
`/var/lib/horizon-grid/backups/`	`pg_dump` snapshots	ProgramData backups
`/var/log/horizon-grid/setup.log`	Setup/runtime log	ProgramData `logs\setup.log`
`/usr/bin/horizon-grid`	CLI entrypoint	--
`/lib/systemd/system/horizon-grid.service`	systemd unit	Windows service registration
`/usr/share/applications/horizon-grid.desktop`	Desktop menu launcher	Start Menu shortcut

Why no AppImage

An AppImage packages a single GUI executable for double-click launch. HORIZON GRID is a multi-container Docker Compose platform accessed via a browser, not a single GUI binary -- there is no one executable to wrap. The `.deb` + systemd + CLI-wizard combination is the direct Linux analog of what the Windows installer already does: neither platform reimplements the application as a native, self-contained desktop program. Docker Compose is the real runtime on both.